

Figure 5 illustrates how different users access modules like appointments, groups, and resources, based on their assigned roles.

Core service modules

We divided our backend into the following functional modules.

Authentication service: Handles user registrations login, and secure session management. Every authenticated request runs through middleware that checks JWT validity and the user's role. Passwords are hashed using Bcrypt before being stored.

Appointment service: This is where patients book sessions and doctors manage their availability. We added real-time updates here using WebSockets, so appointment status changes can be pushed instantly to both sides.

Group service: Support groups are a key part of SereneCare. This module allows doctors and admins to create groups, add members, and oversee discussions. We structured groups so that each has a unique access policy (public/private) and role-based permissions for posting or moderating.

Resource service: This module manages all the health content shared by doctors—whether it's documents, guides, or links. We used Multer for file uploads, and stored metadata like file type, upload date, and category for easy retrieval.

One unexpected benefit of this setup was how much easier it became to test and update individual features. For example, when we needed to change how appointment rescheduling worked, we didn't have to touch anything in the group or resource modules. This isolation of logic made code reviews simpler and allowed contributors to focus on specific services.

It also made it easier to simulate microservices later if we ever needed to deploy modules independently, say, if one service had to scale separately due to load.

SereneCare's modular design wasn't

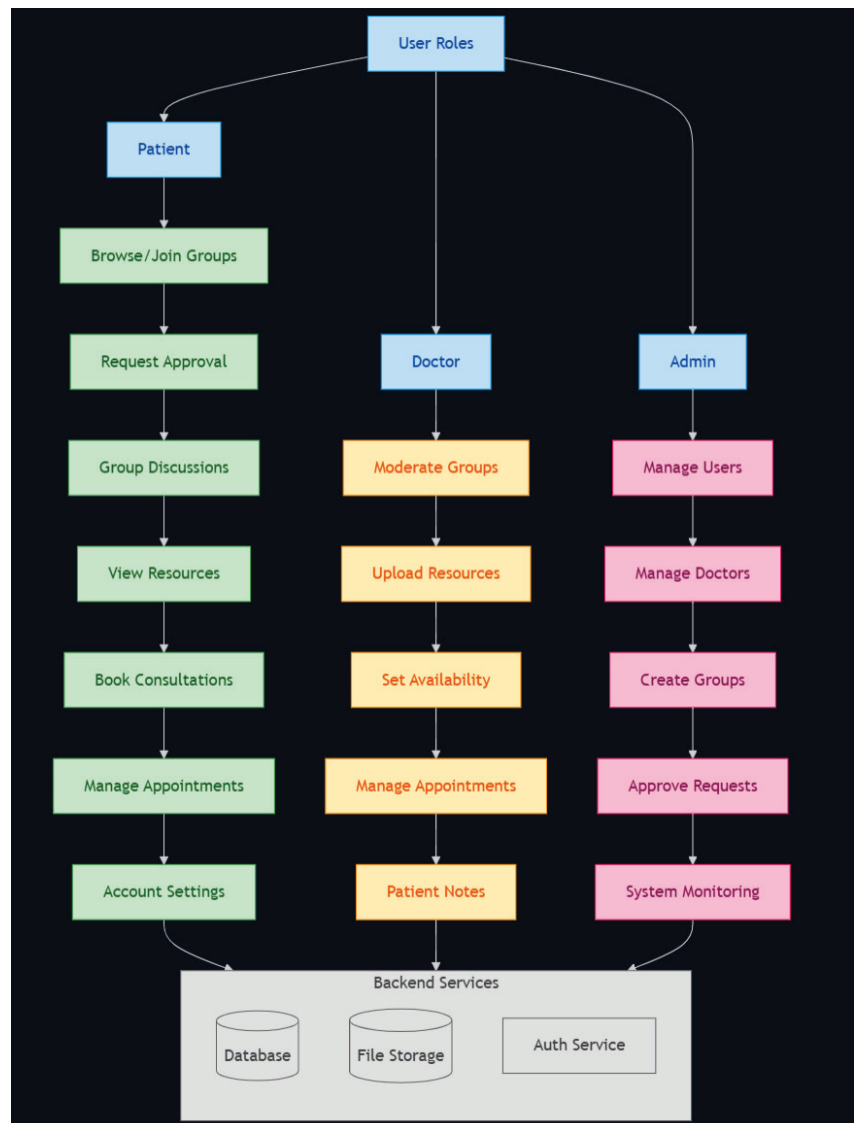


Figure 5: Role-based interaction map

just an architectural decision — it was a development survival tactic. It allowed us to move fast without losing control of the codebase, and it set a foundation that can support future features like analytics or video consultations without needing a complete rewrite.

Data modelling

With a platform like SereneCare, choosing the right data structure was one of the first challenges we had to tackle. Unlike traditional CRUD apps, healthcare platforms deal with a mix of

user roles, sensitive data, appointment metadata, and dynamic content like group discussions or resource uploads.

We chose MongoDB for our backend database, mostly because of its flexible document structure. It allowed us to make quick schema changes during development and model real-world relationships without being locked into rigid SQL schemas.

We kept our data models simple and focused on the actual workflows of the platform. Here are the main ones.

User model: Every user, whether